

Programming Assignment: Dining Philosophers Problem in C

Objective

This assignment introduces students to **multithreading, synchronization, and deadlock avoidance** in C. Students will design a multithreaded system to simulate the **Dining Philosophers Problem**, ensuring correctness, efficiency, and fairness in resource allocation.

Problem Description

The **Dining Philosophers Problem** is a classic concurrency problem formulated by Edsger Dijkstra.

- There are **five philosophers** seated around a circular table.
- Each philosopher alternates between **thinking** and **eating**.
- Between each pair of philosophers lies a single **chopstick** (shared resource).
- To eat, a philosopher needs **both chopsticks** (the one on their left and right).
- After eating, the philosopher puts down both chopsticks and resumes thinking.

Constraints

- No two philosophers can use the same chopstick at the same time.
- The system must avoid **deadlock** (no philosopher should starve forever).
- The solution must ensure **fairness** (each philosopher eventually gets a chance to eat).

Assignment Tasks

1. Multithreading Setup

- Use **POSIX threads (pthread library in C)** to represent philosophers.
- Each philosopher is a thread running an infinite loop of:
 1. Thinking
 2. Attempting to pick up chopsticks
 3. Eating
 4. Putting down chopsticks

2. Synchronization

- Use **mutexes** to represent chopsticks.
- Ensure no two philosophers hold the same chopstick simultaneously.

3. Deadlock Handling

- Implement the following **strategies** to avoid deadlock:
 - Solution A: Number chopsticks and force each philosopher to pick the lower-numbered chopstick first.
 - Solution B: Allow only four philosophers to sit at the table at once.
 - Solution C: Use a **waiter process** (centralized arbiter) to control chopstick allocation.

4. Simulation Output

- The program must print the **state of each philosopher** in real time:
 - Thinking
 - Hungry (waiting for chopsticks)
 - Eating
- Example output format:

```
Philosopher 1 is Thinking
Philosopher 2 is Hungry
Philosopher 2 picked up left chopstick
Philosopher 2 picked up right chopstick
Philosopher 2 is Eating
Philosopher 2 put down chopsticks
```

5. Termination Condition

- Run the simulation for a fixed number of cycles (e.g., each philosopher eats 3 times) before terminating.
- Display summary statistics: number of times each philosopher ate.

Submission Guidelines

- Submit a single C file named `dining_philosophers.c`.
- The program must compile with `gcc -pthread dining_philosophers.c -o dining_philosophers`.
- Code should follow **good programming practices**:
 - Proper indentation
 - Modular structure (functions for thinking, eating, picking, putting chopsticks)
 - Comments explaining the synchronization logic